МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

ИНСТИТУТ ИНФОРМАЦИОННЫХ СИСТЕМ И ЗАЩИТЫ ИНФОРМАЦИИ

КАФЕДРА № 43

ОТЧЕТ ЗАЩИЩЕН С ОЦЕНКОЙ	ПРЕПОДАВАТЕЛЬ	ассистент А. Э. Зянчурин
	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 1

«Автоматизация технологического процесса промышленного производства программного обеспечения»

по дисциплине: «Основы программной инженерии»

Индивидуальный раздел: «Архитектор программного обеспечения. Проектирование архитектуры системы»

РАБОТУ ВЫПОЛНИЛ	СТУДЕНТ гр. № 4434	Прохоров М. Ю.
	подпись, дата	инициалы, фамилия

Санкт-Петербург

2026

ЗАДАНИЕ

На индивидуальное выполнение раздела лабораторной работы студентом группы 4434 Прохоров М. Ю..

Тема задания: Проектирование архитектуры программного обеспечения информационной системы супермаркета в роли архитектора ПО

Перечень подлежащих исследованию, проектированию и разработке вопросов:

— Обоснование трехуровневой архитектурной структуры (уровень представления UI, уровень прикладных сервисов API, уровень хранения данных СУБД) и архитектурного паттерна модульного монолита.

— Декомпозиция программной системы на функциональные модули (Каталог товаров, Складской учет, Кассовые чеки, Аутентификация и безопасность) и спецификация межмодульных контрактов.

— Проектирование транзакционной модели кассовой продажи с обеспечением атомарного списания остатков и защиты от race condition через блокировку строк.

— Разработка структуры физического размещения компонентов (Deployment-диаграмма) с учетом контейнеризации и сетевой изоляции.

— Разработка структуры реализации программного обеспечения: отображение логических модулей на файловую структуру кодовой базы.

— Анализ надежности, отказоустойчивости, изоляции секретов и эксплуатационных границ архитектуры.

Задание выдал: _____ / А. Э. Зянчурин /
Подпись: _____
Задание принял: _____ / Прохоров М. Ю. /
Дата выдачи: «____» _____ 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ

1 АРХИТЕКТУРНЫЙ СТИЛЬ И ОБОСНОВАНИЕ РЕШЕНИЙ

1.1 Обоснование трехуровневой архитектуры

1.2 Обоснование паттерна модульного монолита

2 МОДУЛЬНАЯ СТРУКТУРА И ИНТЕРФЕЙСЫ ВЗАИМОДЕЙСТВИЯ

2.1 Декомпозиция системы на модули

2.2 Модули, методы в псевдокоде и зависимости

2.3 Соединители и протоколы обмена

3 ТРАНЗАКЦИОННАЯ МОДЕЛЬ И СОГЛАСОВАННОСТЬ ДАННЫХ

3.1 Алгоритм атомарного проведения чека и списания остатков

3.2 Защита от параллельных гонок кассиров (Race Condition)

3.3 Идемпотентность и двухфазная смена состояния

4 СТРУКТУРЫ РАСПРЕДЕЛЕНИЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

4.1 Структура физического размещения (Deployment-диаграмма)

4.2 Требования к ресурсам стенда виртуализации

4.3 Структура реализации и план отображения модулей на файлы

5 НАДЕЖНОСТЬ, ОТКАЗОУСТОЙЧИВОСТЬ И БЕЗОПАСНОСТЬ

5.1 Обработка сбоев канала связи и таймауты запросов

5.2 Управление секретами и изоляция учетных записей

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

ВВЕДЕНИЕ

Настоящий отчет представляет результаты архитектурного проектирования информационной системы розничного супермаркета (вариант 36) в рамках лабораторной работы № 1 по дисциплине «Основы программной инженерии».

Главной задачей архитектора программного обеспечения является определение высокоуровневой структуры системы, декомпозиция на слабосвязанные модули, выбор архитектурного паттерна, обоснование используемых протоколов взаимодействия, а также проектирование структур размещения и реализации, гарантирующих выполнение жестких требований к транзакционной целостности и надежности.

Проектирование выполнено с учетом стандартов ISO/IEC/IEEE 42010 и лучших практик архитектурного анализа программных комплексов.

1 АРХИТЕКТУРНЫЙ СТИЛЬ И ОБОСНОВАНИЕ РЕШЕНИЙ

1.1 Обоснование трехуровневой архитектуры

В качестве основополагающей архитектурной модели выбрана классическая трехуровневая архитектура (Three-tier Architecture), включающая:

— Уровень представления (Presentation Tier) — веб-клиент на базе современного компонентного фреймворка, выполняемый в браузере на автоматизированном рабочем месте сотрудника;

— Уровень прикладных сервисов и бизнес-логики (Application Tier) — сервер приложений на базе высокопроизводительного асинхронного веб-фреймворка, обеспечивающий строгую валидацию входных данных, авторизацию и оркестрацию бизнес-правил;

— Уровень постоянного хранения данных (Data Tier) — реляционная СУБД с гарантией поддержки требований ACID, обеспечивающая сохранность номенклатуры, кассовых проводок и управление транзакциями.

1.2 Обоснование паттерна модульного монолита

В качестве архитектурного паттерна для уровня бизнес-логики выбран модульный монолит (Modular Monolith). Данное решение обосновано следующими факторами:

— Атомарность кассовой транзакции: проведение чека и синхронное уменьшение остатка по нескольким позициям требуют выполнения в рамках единой локальной транзакции базы данных. При микросервисном разделении возникла бы необходимость распределенных двухфазных коммитов (2PC) или сложных саг (Saga), что неоправданно усложняет учебную систему и снижает надежность;

— Размер команды и сложность развертывания: модульный монолит позволяет вести строгую модульную изоляцию на уровне интерфейсов и директорий, исключая сетевые накладные расходы на сериализацию между внутренними модулями;

— Прозрачность тестирования: модули легко тестируются сквозными интеграционными тестами в едином процессе.

2 МОДУЛЬНАЯ СТРУКТУРА И ИНТЕРФЕЙСЫ ВЗАИМОДЕЙСТВИЯ

2.1 Декомпозиция системы на модули

Система разделена на четыре автономных функциональных модуля:

— Модуль каталога (Catalog Module) — поиск, фильтрация, карточки товаров, цены и штрихкоды;

— Модуль складского учета (Stock Module) — операции оприходования, списания и актуализация остатков;

— Модуль кассовых чеков (Checkout Module) — создание черновиков чеков, наполнение позициями, расчет сумм, проведение оплаты наличными;

— Модуль аутентификации и безопасности (Auth & Security Module) — проверка учетных записей, управление токенами сессий и разграничение прав.

2.2 Модули, методы в псевдокоде и зависимости

В таблице 1 представлена детальная спецификация модулей системы, их методов и зависимостей.

Таблица 1 — Модули, методы и данные проектируемой системы

Модуль	Назначение и методы в псевдокоде	Данные и зависимости
Auth	login(credentials), me(token), logout(token), requireRole(role)	Сотрудник, сеанс; использует Security и DB
Security	hashPassword(password), verifyPassword(password, hash), hashToken(token)	Соль и хеш; не зависит от UI
Catalog	list(query,cursor,limit), create(card), update(card,version)	Товар; использует Auth, Validation, DB
Inventory	adjust(product,delta,reason), recordMovement(...)	Остаток и движение; использует Auth, DB
Sales	createReceipt(key,lines), pay(id), cancel(id), read(id), history(cursor)	Чек, строка; использует Auth, Catalog/Inventory через общую транзакцию
Validation	validateProduct(...), validateReceipt(...)	Схемы запросов; ограничивает типы, диапазоны и повтор товаров
DB	acquireConnection(), begin(), commit(), rollback()	Пул соединений и параметризованные SQL-запросы
Web Catalog	search(), nextPage(), addToCart(), editCard()	Состояние фильтра, текущая страница; вызывает API-client
Web Checkout	prepare(), confirmCash(), cancelDraft()	Корзина, серверный чек, ключ повтора
API-client	request(path,body,token), validateResponse(...)	JSON, отмена запроса, преобразование ошибок

2.3 Соединители и протоколы обмена

Взаимодействие между клиентом и сервером приложений стандартизировано на протоколе HTTP/1.1 с передачей полезной нагрузки в формате JSON (RESTful API). Защищенные маршруты требуют передачи заголовка Authorization: Bearer <TOKEN>.

Взаимодействие сервера приложений с СУБД осуществляется через оптимизированный пул соединений (Connection Pooling). Все параметры подключения изолируются в переменных окружения.

3 ТРАНЗАКЦИОННАЯ МОДЕЛЬ И СОГЛАСОВАННОСТЬ ДАННЫХ

3.1 Алгоритм атомарного проведения чека и списания остатков

Для обеспечения строгой целостности данных кассовая операция разделена на две фазы:

— Фаза 1: формирование черновика (draft). Чек сохраняется в БД со связями на позиции, но остатки товаров не блокируются и не списываются. Это позволяет кассиру свободно редактировать корзину;

— Фаза 2: подтверждение оплаты. Выполняется в рамках единой неделимой транзакции уровня изоляции READ COMMITTED.

3.2 Защита от параллельных гонок кассиров (Race Condition)

При параллельной работе нескольких кассовых узлов возможна ситуация одновременной продажи последнего экземпляра товара. Для устранения коллизий применяется механизм пессимистической блокировки строк на уровне СУБД:

```
BEGIN TRANSACTION;
SELECT id, stock FROM products WHERE id = :item_id FOR UPDATE;
-- проверка: если stock < requested_qty, генерируется ROLLBACK и ошибка 409 Conflict
UPDATE products SET stock = stock - :requested_qty WHERE id = :item_id;
INSERT INTO stock_movements ...;
UPDATE receipts SET status = 'paid', paid_at = NOW() WHERE id = :receipt_id;
COMMIT;
```

Блокировка FOR UPDATE гарантирует, что вторая касса дождется завершения транзакции первой и получит актуальный нулевой остаток, предотвратив отрицательные остатки.

3.3 Идемпотентность и двухфазная смена состояния

Запросы на оплату чека проектируются идемпотентными: если клиент из-за сетевого сбоя повторно отправляет запрос на оплату уже проведенного чека, сервер возвращает статус 200 OK с ранее зафиксированными данными, не производя повторного списания остатков со склада.

4 СТРУКТУРЫ РАСПРЕДЕЛЕНИЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

4.1 Структура физического размещения (Deployment-диаграмма)

Физическая топология системы разработана в соответствии со стандартами промышленного развертывания.

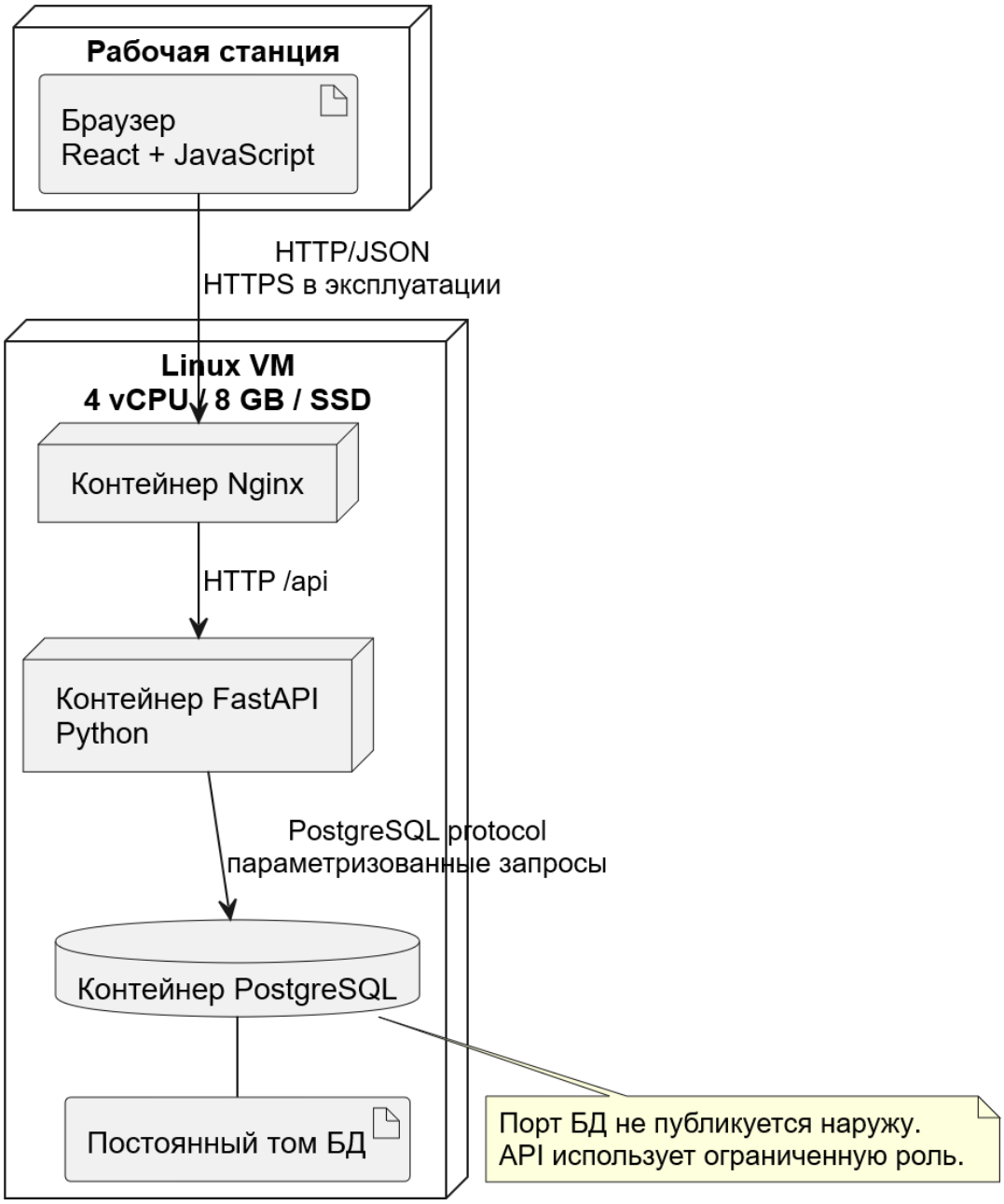


Рисунок 1 — Диаграмма физического размещения компонентов системы (Deployment Diagram)

Топология включает:

— Клиентские рабочие места кассиров и менеджеров: обычные ПК с современными браузерами, подключающиеся к веб-серверу по локальной сети супермаркета;

— Серверный контур: выделенная виртуальная машина Linux, содержащая контейнеры Nginx (веб-сервер и обратный прокси), FastAPI (сервер приложений) и PostgreSQL (сервер СУБД);

— Сетевая безопасность: база данных и сервер приложений не публикуют открытые сетевые порты во внешнюю сеть, взаимодействие происходит внутри защищенной внутренней сети.

4.2 Требования к ресурсам стенда виртуализации

Для развертывания проектного стенда определены требования к ресурсам виртуальной машины:

— Процессор: не менее 4 vCPU;

— Оперативная память: не менее 8 ГБ RAM;

— Дисковое пространство: 40 ГБ с динамическим расширением;

— Сетевые адаптеры: NAT для загрузки пакетов и Host-Only для изоляции трафика разработки.

4.3 Структура реализации и план отображения модулей на файлы

В таблице 2 представлен детальный план декомпозиции архитектурных модулей на структуру каталогов и файлов будущей реализации.

Таблица 2 — План отображения модулей на файлы будущей реализации

Модуль	**Планируемое место реализации на этапе ЛР №3**
Сборка приложения и lifecycle	backend/app/main.py
HTTP-маршруты Auth, Catalog, Sales	backend/app/routers/auth.py, products.py, receipts.py
Предметные операции Catalog, Inventory, Sales	backend/app/services/products.py, receipts.py
Аутентификация и проверка ролей	backend/app/dependencies.py
Преобразование ошибок в HTTP-ответы	backend/app/errors.py
Validation	backend/app/schemas.py
Security	backend/app/security.py
DB	backend/app/db.py
Схема и права БД	init.sql, db/pg_hba.conf
Web Catalog	frontend/src/Catalog.jsx
Оркестрация состояния интерфейса	frontend/src/App.jsx
Вход, навигация, редактор товара	frontend/src/components/*.jsx
Checkout и история	frontend/src/features/checkout/*.jsx, frontend/src/features/receipts/*.jsx

API-client	frontend/src/api.js
Стили	frontend/src/styles.css
Развёртывание прикладной версии	docker-compose.yml, Dockerfile компонентов

5 НАДЕЖНОСТЬ, ОТКАЗОУСТОЙЧИВОСТЬ И БЕЗОПАСНОСТЬ

5.1 Обработка сбоев канала связи и таймауты запросов

В архитектуре предусмотрена строгая обработка непредвиденных сбоев:

— Таймауты клиентских запросов составляют 5 секунд, предотвращая зависание интерфейса кассира;

— При кратковременной недоступности сервера клиент отображает статус ошибки и сохраняет текущее состояние корзины в оперативной памяти страницы;

— Пул соединений с БД имеет лимит на максимальное число одновременных подключений (до 20 соединений), защищая СУБД от перегрузки.

5.2 Управление секретами и изоляция учетных записей

Архитектура разграничивает прикладные и технические учетные записи:

— Сервер приложений подключается к БД под выделенной учетной записью market_app, лишенной прав на удаление таблиц (DROP/ALTER);

— Все секреты (пароли, ключи) передаются исключительно через защищенный файл локального окружения .env и не попадают в систему контроля версий;

— Инфраструктурный контур системного управления изолирован от продуктовой базы данных приложения.

ЗАКЛЮЧЕНИЕ

В результате выполнения индивидуального раздела лабораторной работы № 1 в роли архитектора программного обеспечения сформирована законченная архитектурная концепция информационной системы розничного супермаркета.

Были достигнуты следующие проектные результаты:

— Обоснован выбор трехуровневой архитектуры и паттерна модульного монолита, наилучшим образом удовлетворяющего требованиям локальной транзакционной целостности розничной торговли;

— Сформирована модульная структура системы, специфицированы методы, входные параметры и взаимосвязи ключевых модулей;

— Разработана транзакционная модель проведения чека с пессимистической блокировкой строк (FOR UPDATE), гарантирующая отсутствие гонок и защиту от отрицательных остатков;

— Спроектирована физическая структура размещения компонентов (Deployment) и план отображения архитектурных компонентов на структуру файлов кодовой базы.

Разработанная архитектурная спецификация является надежной технической основой для последующей организации сред разработки и практической реализации системы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Охтилев П.А., Зянчурин А.Э. Основы программной инженерии: Методические указания к выполнению лабораторных работ. — СПб.: ГУАП, 2021. — 75 с.

2. ГОСТ 2.105-2019. Единая система конструкторской документации. Общие требования к текстовым документам. — М.: Стандартинформ, 2019. — 38 с.

3. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. — М.: Стандартинформ, 2018. — 32 с.

4. ISO/IEC/IEEE 42010:2011. Systems and software engineering — Architecture description. — Geneva: ISO, 2011. — 46 p.

5. Басс Л., Клементс П., Кацман Р. Архитектура программного обеспечения на практике. — 3-е изд. — СПб.: Питер, 2014. — 576 с.

6. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения. — СПб.: Питер, 2018. — 352 с.